Performance Model and Compute Strategy

Fiber Raman Suppression Project

Evidence snapshot: 2026-04-26

Abstract

This appendix explains how to think about runtime in the Raman-suppression project. The main result is practical: for the canonical single-mode workload, the fastest path is not "turn on more threads inside one solve." The important runtime decisions are to use adjoint gradients instead of finite differences, keep FFTW internally single-threaded at the current grid sizes, and spend parallel compute on independent runs such as sweeps, multistarts, and parameter studies. The benchmark evidence is strong for the canonical single-mode case. Multimode and long-fiber workloads should be remeasured before reusing the same threading conclusions.

Claim Status

Status: established for the canonical single-mode workload
The benchmark data support a stable compute rule for the current SMF-28 single-mode problem: use adjoint gradients, keep FFTW internal threading at one thread, and parallelize across independent jobs when possible. The caveat is important: multimode and long-fiber studies may move the bottleneck, so those directions should be benchmarked separately before making infrastructure decisions.

1 Question and Thesis

The question in this appendix is not whether the physics model is correct. It is where compute effort actually helps.

The thesis is:

$$\text{adjoint gradients} + \text{task-level parallelism} \quad \text{beat} \quad \text{more internal threads for one canonical solve.}$$

That matters because this project has several tempting but expensive paths: full-grid gradients, multistart searches, reduced-basis ladders, second-order diagnostics, multimode runs, and long-fiber runs. A bad performance model would push effort toward the wrong machine or the wrong optimization method.

Intuition Check

If one solve is mostly serial, adding more workers to that one solve does not help much. The better use of workers is to run many independent solves at the same time: different starts, different fiber settings, different basis sizes, or different validation cases.

2 The Cost Model

The baseline objective is the output Raman-band energy fraction,

$$J_{\text{phys}}(\phi) = \frac{\sum_{\omega_k \in \mathcal{B}_{\text{Raman}}} |u_{\omega_k}(L;\phi)|^2}{\sum_k |u_{\omega_k}(L;\phi)|^2},$$

with optional regularization before the dB transform:

$$J_{\text{surf}}(\phi) = J_{\text{phys}}(\phi) + \lambda_{\text{gdd}} R_{\text{gdd}}(\phi) + \lambda_{\text{edge}} R_{\text{edge}}(\phi), \qquad F(\phi) = 10 \log_{10} J_{\text{surf}}(\phi).$$

The phase vector can have thousands of entries. In the canonical grid, $N_t = 8192$, so a naive central-difference gradient would cost roughly

$$2N_t\, C_{\text{fwd}} = 16384\, C_{\text{fwd}},$$

where C_{fwd} is the cost of one forward fiber solve. That is not a reasonable inner loop for L-BFGS.

The adjoint method changes the scaling. One cost-and-gradient evaluation is approximately

$$C_{\text{grad}} \approx C_{\text{fwd}} + C_{\text{adj}} + C_{\text{regularizer}},$$

where C_{adj} is the backward sensitivity solve. The important point is that this cost does not grow like the number of phase pixels.

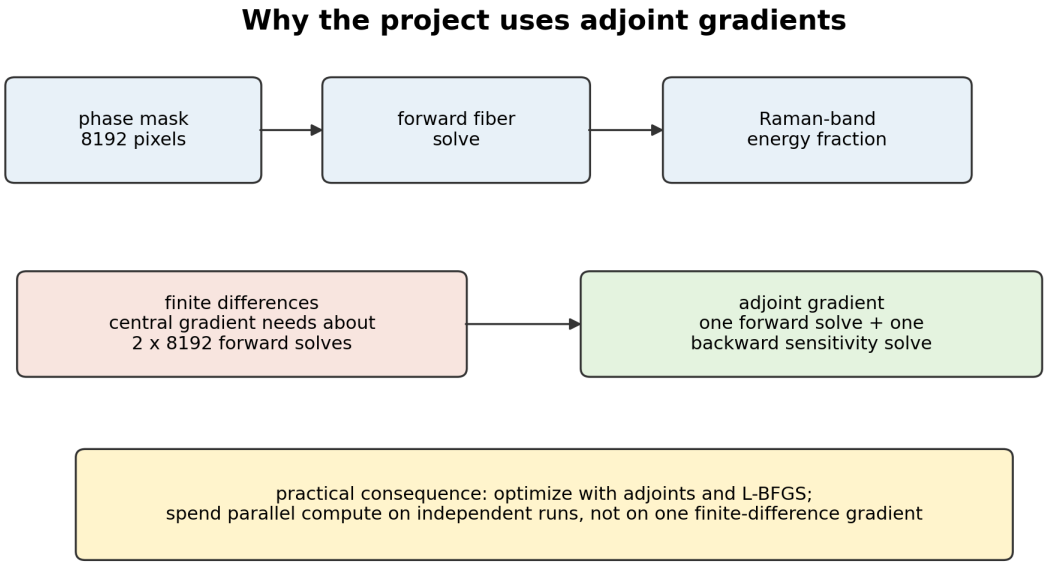

Figure 1: Why adjoints are the default gradient method. Finite differences ask one question per phase pixel. The adjoint solve gives the whole phase-gradient direction in one backward sensitivity pass.

Intuition Check

Finite differences are like changing every knob one at a time and rerunning the whole experiment. The adjoint is like one backward calculation that tells us which way all knobs should move.

3 Measured Runtime Evidence

The canonical benchmark used an SMF-28 single-mode setting with $L = 2\,\mathrm{m}$, $P = 0.2\,\mathrm{W}$, $N_t = 8192$, Julia 1.12.4, and an Apple M3 Max host. The exact numbers should not be treated as universal hardware constants. The shape of the result is the important part.

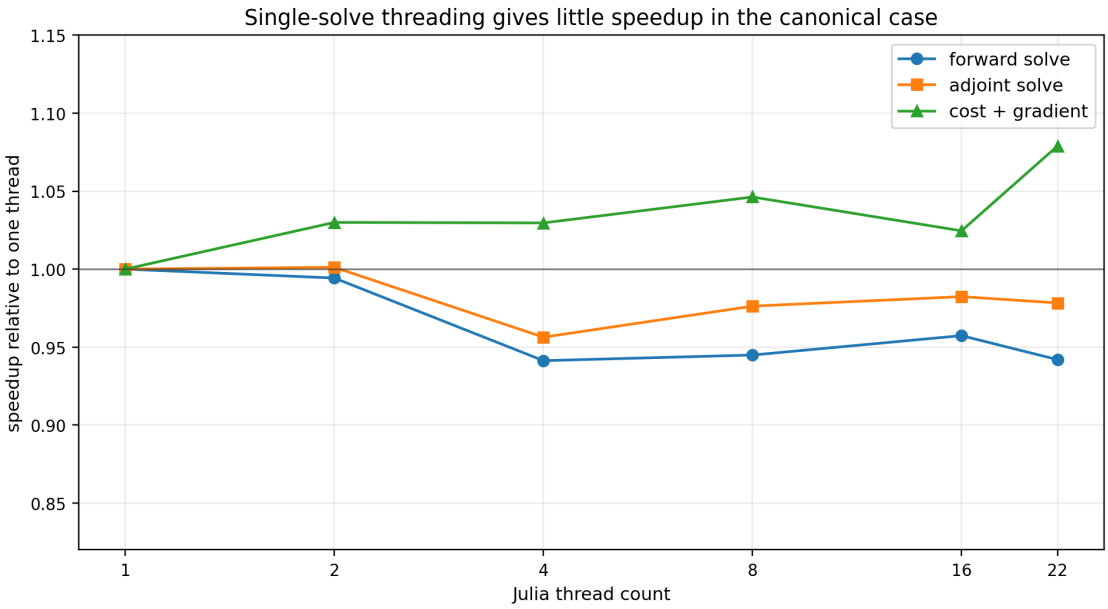

Figure 2: Single-solve threading speedup relative to one Julia thread. Forward and adjoint solves do not improve; the full cost-and-gradient call improves only weakly. This is the measured reason the project does not treat internal thread count as the main lever for the canonical single-mode workload.

Measured mode	**One thread**	**Best observed**	**Fitted ceiling**	**Takeaway**
Forward solve	1.178 s	1.178 s	1.00×	Extra Julia threads do not speed up the forward solve.
Adjoint solve	1.325 s	1.323 s	1.00×	The measured difference is noise-level for planning.
Cost + gradient	2.708 s	2.510 s	1.05×	There is a small gain, but not enough to justify a large single-job machine for this case.

The kernel-level timing tells the same story. The adjoint right-hand side is materially heavier than the forward right-hand side, and the raw FFT timings do not explain the whole runtime.

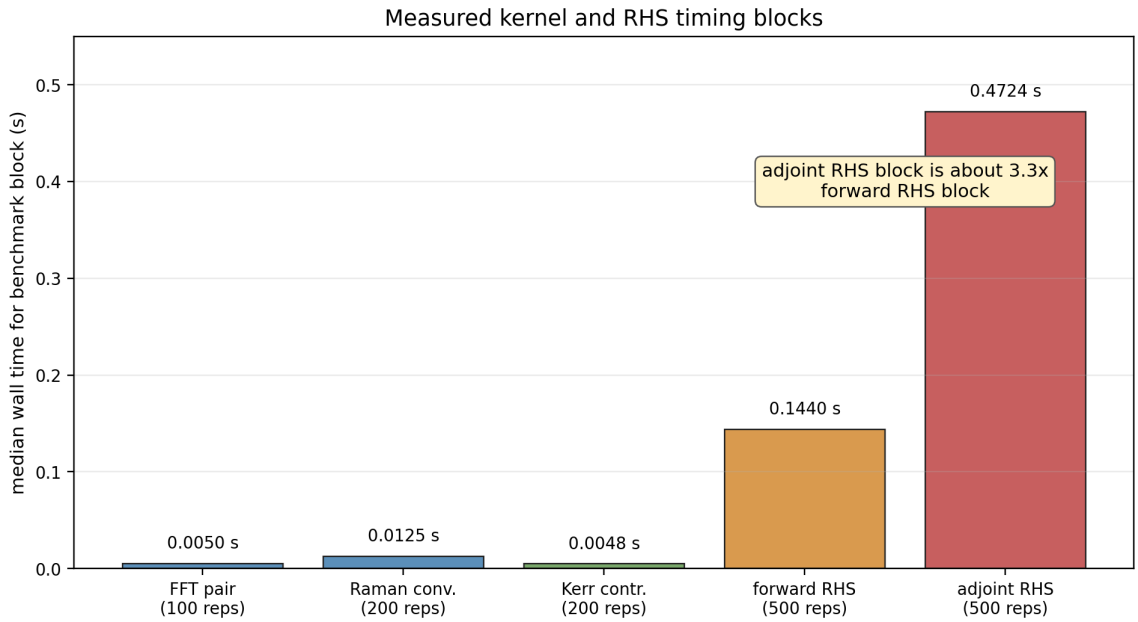

Figure 3: Representative timing blocks from the benchmark artifact. The adjoint right-hand-side block is about 3.3× the forward right-hand-side block, so gradient evaluations are much more than "one forward solve with a small extra step."

Block	Repetitions	Median time	Interpretation
FFT forward + inverse pair	100	0.0050 s	Single-thread FFT planning is fast at this grid size.
Raman frequency convolution	200	0.0125 s	Raman terms are visible but not the only cost.
Kerr tensor contraction	200	0.0048 s	Trivial in this single-mode case; important again in multimode.
Forward RHS block	500	0.1440 s	About 288 μs per call.
Adjoint RHS block	500	0.4724 s	About 945 μs per call, partly due to forward-solution interpolation inside the adjoint.

Intuition Check

The adjoint is still the right method because it replaces thousands of forward solves. But it is not free. When a note says "one gradient evaluation," read that as a forward solve plus a heavier backward sensitivity solve.

4 Why FFTW Internal Threading Is Not the Lever

At $N_t = 8192$, internal FFTW threading is counterproductive in the benchmark. The one-thread FFT pair block took 0.0050 s. The best multi-thread FFT pair block took 0.0822 s, about a 16.3× slowdown for that isolated FFT test.

The likely reason is overhead. The FFTs at this grid size are not large enough to pay for the extra thread coordination inside each transform. This is why the current rule is:

`FFTW.set_num_threads(1)` unless a new workload is remeasured.

There is also a reproducibility tradeoff. Deterministic FFT planning was measured to be slower than timed plan selection, but the deterministic path keeps independent optimizer runs comparable.

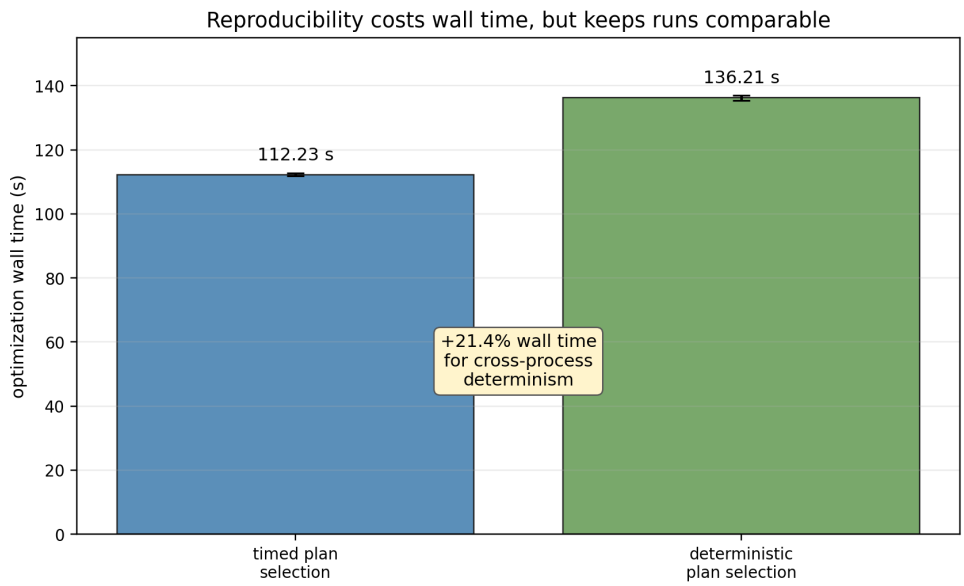

Figure 4: Deterministic FFT planning costs wall time in the benchmark, but it removes a source of cross-process plan-selection variation. The measured slowdown was 21.4%, inside the project's accepted reproducibility budget.

Intuition Check

There are two separate threading questions. Julia threads control how much parallel work the program can schedule. FFTW threads control how many workers FFTW uses inside one transform. For this grid, FFTW's internal threads make the FFT block slower, not faster.

5 Parallelism Strategy

Amdahl's law says that if a fraction p of one job is parallelizable, then the best possible strong-scaling speedup on N workers is

$$S_{\text{Amdahl}}(N) = \frac{1}{(1-p) + p/N}.$$

For the canonical single-mode benchmark, the fitted p values were effectively zero for forward and adjoint solves, and about 0.05 for the full cost-and-gradient call. That gives a speedup ceiling near 1.0×, or only about 1.05× for the full gradient call.

That does not mean parallel compute is useless. It means the better scaling surface is weak scaling across independent jobs:

$$\text{many independent starts or sweep points} \quad \Rightarrow \quad \text{many cores can stay busy.}$$

Low-leverage use of cores	**High-leverage use of cores**
More internal FFTW threads for one canonical solve	Many independent sweep points
Buying a large VM for one single-mode cost-and-gradient call	Many multistart runs with separate random seeds
Assuming a multimode bottleneck from single-mode timings	Remeasuring multimode kernels before choosing the machine

Intuition Check

One hard-to-parallelize job is like one checkout line with many idle cashiers. Many independent runs are like opening many checkout lines at once.

6 Compute Discipline

The practical machine rule is:

- local or orchestration hosts are appropriate for editing, plotting, PDF generation, indexing, and light checks;
- heavy simulation campaigns belong on the burst machine through the approved wrapper;
- Julia simulation runs should be launched with threading enabled, but FFTW internal threading should remain at one thread unless the workload has been remeasured;
- the burst machine is most useful for multimode studies, long sweeps, multistarts, and other embarrassingly parallel jobs;
- stop the burst machine when the heavy job is complete.

For the current canonical single-mode case, a large high-CPU VM is not justified for one optimization run. It can still be justified if the job list contains many independent runs.

7 Multimode and Long-Fiber Caveat

This appendix should not be read as "performance is solved." It is a scoped single-mode performance model.

For $M = 1$, FFT-heavy reasoning is often a decent first approximation. For $M \geq 3$, the Kerr tensor contractions and memory traffic can dominate the right-hand side. Long-fiber runs may also change runtime through solver step counts, stiffness, and output checkpointing. Those cases need their own timing artifacts before making strong claims.

Intuition Check

Changing from one mode to many modes is not just making the same calculation a little bigger. It changes the shape of the work. The benchmark target has to match the research target.

8 Image Rule for This Appendix

Most notes in this series pair phase diagnostics with spectral-evolution heat maps. This appendix is different: it does not introduce a new optimized optical field. Its figures are therefore benchmark diagrams and timing charts rather than phase-profile pages. Optical-result notes should still include the paired phase-diagnostic and heat-map pages.

9 Presentation Capsule

If this appendix becomes a compute-methods slide, the clean message is:

- The adjoint is what makes full-grid optimization feasible; finite-differencing thousands of phase pixels is not practical.
- More threads inside one solve are not automatically the best speedup path; workload shape and run-level parallelism matter.
- Deterministic numerics can be worth a small runtime cost because optimizer comparisons need reproducible surfaces.
- This is a support slide: use it to explain how the research was run responsibly, not as a primary physics-result slide.

10 Reproduction Capsule

Benchmark commands should be run only when the machine and workload match the question being asked:

```
julia -t auto -project=.  scripts/research/benchmarks/benchmark_threading.jl

julia -t auto -project=.  scripts/workflows/run_benchmarks.jl
```

For a new benchmark, record at least:

- Julia version, host CPU, memory, Julia thread count, BLAS thread count, and FFTW thread count;
- fiber, length, power, grid size, number of modes, time window, and solver tolerances;
- forward solve timing, adjoint solve timing, full cost-and-gradient timing, and any isolated kernel timing used to justify a conclusion;
- whether the benchmark is single-mode, multimode, long-fiber, or a sweep/multistart workload.

11 Key Lessons and Limitations

Use adjoints. Full-grid finite differences are not viable for routine optimization.

Do not overbuy single-job threads. The canonical single-mode cost-and-gradient call has only a small measured strong-scaling gain.

Keep deterministic numerics visible. A slower deterministic FFT plan can be the right choice if it keeps optimizer comparisons trustworthy.

Parallelize the experiment list. Sweeps, multistarts, basis ladders, and validation batches are where parallel compute gives the cleanest return.

Remeasure outside the canonical case. Multimode and long-fiber studies can have different bottlenecks, so their compute plans should be based on their own benchmark artifacts.

12 Sources and References

References

[1] M. Frigo and S. G. Johnson. "The design and implementation of FFTW3." *Proceedings of the IEEE*, 93(2), 216–231, 2005. https://doi.org/10.1109/JPROC.2004.840301

[2] G. M. Amdahl. "Validity of the single processor approach to achieving large scale computing capabilities." *AFIPS Conference Proceedings*, 30, 483–485, 1967. https://doi.org/10.1145/1465482.1465560

[3] J. L. Gustafson. "Reevaluating Amdahl's law." *Communications of the ACM*, 31(5), 532–533, 1988. https://doi.org/10.1145/42411.42415

[4] M. B. Giles and N. A. Pierce. "An introduction to the adjoint approach to design." *Flow, Turbulence and Combustion*, 65, 393–415, 2000. https://doi.org/10.1023/A:1011430410075

[5] A. Griewank and A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. SIAM, 2008. https://doi.org/10.1137/1.9780898717761

[6] D. C. Liu and J. Nocedal. "On the limited memory BFGS method for large scale optimization." *Mathematical Programming*, 45, 503–528, 1989. https://doi.org/10.1007/BF01589116

[7] J. Nocedal and S. J. Wright. *Numerical Optimization*, 2nd ed. Springer, 2006. https://doi.org/10.1007/978-0-387-40065-5

[8] G. P. Agrawal. *Nonlinear Fiber Optics*, 6th ed. Academic Press, 2019. https://www.sciencedirect.com/book/9780128170427/nonlinear-fiber-optics